

Solution Requirements

Date	15 March 2026
Team ID	EDUGENIE-01
Project Name	EduGenie – AI-Powered Educational Assistant
Maximum Marks	4 Marks
Team Lead	Pavan Varma Uddaraju
Members	Voruganti Krishna Sanket Naresh C Humaira Tasneem Syed Sai Subash Raja Kosaraju

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should *do*).

Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Users access EduGenie directly via the web interface — no login required for basic usage. Optional session-based authentication can be added for saving progress and personalizing learning history.	Medium
2	Authorization Levels	Two roles: Guest User (access to Q&A, quiz, summarize, learning path features) and Admin (manage API keys, monitor usage, update system configurations).	Medium
3	External Interfaces	Google Gemini API (generative AI responses), SQLite/PostgreSQL (data storage), FastAPI backend (REST endpoints: /qa, /explain, /quiz, /summarize, /learn/recommendations).	High
4	Transaction Processing	Each user query is processed as a stateless transaction via FastAPI. Input is sent to the Gemini API, the response is received and formatted, then returned to the frontend in real time.	High
5	Reporting	Session-based logs of user queries and AI responses. Admin dashboard to view usage metrics (queries per module, popular topics, error rates). Exportable as CSV.	Low

6	Business Rules	Each query must include a topic and mode (Q&A; / Explain / Quiz / Summarize / Learn Path). Quiz must generate minimum 5 questions. Learning path must cover beginner, intermediate, and advanced levels.	High
7	Compliance to Laws / Regulations	No personally identifiable information (PII) stored without consent. API keys stored securely as environment variables. Compliant with general data privacy best practices (GDPR-aligned).	High
8	Other	System must support multi-subject queries (Science, Math, Programming, History, etc.). Responses must be in plain English suitable for students at different academic levels.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The system must respond to user queries quickly. FastAPI with async processing ensures low-latency communication with the Gemini API.	API response delivered to frontend in < 3 seconds for 95% of requests.
2	Scalability	The FastAPI backend must handle concurrent users efficiently. SQLite for development; PostgreSQL for production scale.	Supports up to 500 concurrent users; scales horizontally with Docker containers.
3	Security & Data Privacy	Gemini API keys stored as environment variables (not hardcoded). Input sanitization to prevent prompt injection. HTTPS enforced in production.	Zero exposed API keys in codebase. All data transmitted over HTTPS/TLS.
4	Reliability & Availability	System should remain available during peak academic periods. Graceful error handling when Gemini API is unavailable, with fallback messages.	99.5% uptime per month. Errors returned within 2 seconds with user-friendly messages.
5	Usability & Accessibility	Clean, intuitive HTML+CSS interface usable on desktop and mobile. Clear labels, readable fonts, and simple navigation for students of all levels.	Usable on screens ≥ 320px. Complies with WCAG 2.1 AA accessibility standards.
6	Other (Maintainability & Portability)	Code structured in modular FastAPI routers (one per feature). Well-documented with docstrings. Runs on Windows, Linux, and macOS (Mac M1 optimized).	New modules deployable with < 1 hour of setup. Runs on Python 3.10+ across all major OS platforms.